

AI Hiring Assistant

An end-to-end AI Hiring Assistant platform built with **Next.js (Frontend)**, **FastAPI (Backend)**, and the **Hunar AI Voice Platform**.

This application allows you to create Job descriptions, upload Candidates, and automatically trigger an AI Voice Agent to call candidates on their real phones. The AI will interview them based on the job requirements, record the call, and extract structured insights (like Expected CTC, Notice Period, and overall Fit Summary) back into your dashboard.

🏗️ Architecture

- **Frontend:** Next.js (React), Tailwind CSS, shadcn/ui. Runs on `http://localhost:3000`.
- **Backend:** FastAPI (Python), SQLite (Local Database), SQLAlchemy. Runs on `http://localhost:8000`.
- **Background Worker:** A standalone Python polling script (`polling_worker.py`) that syncs call statuses from the Hunar API down to the local SQLite database.

🚀 Step 1: Backend Setup (FastAPI)

1. **Navigate to the backend directory:**

```
cd backend
```

2. **Create a Python Virtual Environment & Install Dependencies:**

```
python -m venv venv
.\venv\Scripts\activate # (On Windows)
pip install fastapi uvicorn sqlalchemy httpx python-dotenv
```

3. **Configure the Environment File:**

Make sure you have a `.env` file inside the backend folder containing your Hunar API Key:

```
HUNAR_API_KEY=hunar_va_live_sk_YOUR_API_KEY_HERE
DATABASE_URL=sqlite:///./test.db
```

(Note: Because we are using a background polling worker, `PUBLIC_BACKEND_URL` is no longer strictly required for local development!)

4. **Start the Backend Server:**

```
python -m uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
```

🔄 Step 2: Start the Background Polling Worker

Because local webhooks can be tricky (due to tunneling tools like `localtunnel` getting blocked by password screens), we built a **Polling Worker** that automatically fetches live call results and updates the dashboard reliably.

1. Open a **new terminal tab**.
2. Navigate to the backend directory and activate your virtual environment:

```
cd backend
.\venv\Scripts\activate
```

3. Run the polling script:

```
python polling_worker.py
```

Leave this running in the background! It will check the Hunar API every 5 seconds for updates on active calls.

🖥️ Step 3: Frontend Setup (Next.js)

1. Open a **new terminal tab**.
2. **Navigate to the frontend directory:**

```
cd frontend
```

3. **Install Dependencies:**

```
npm install
```

4. **Start the Frontend Server:**

```
npm run dev
```

🎧 How to Use the App

1. **Open the App:**
Go to <http://localhost:3000> (<http://localhost:3000>) in your web browser.
2. **Create a Job:**
Click **Create Job**. Give it a title (e.g., "Frontend Developer") and list the key requirements (e.g., "React, Next.js, 3 years experience"). The backend will dynamically create a custom AI Agent on the Hunar platform tailored to this job!
3. **Add Candidates:**
Click **Manage Candidates** on your new job. Add a candidate by entering their name and **your real phone number** (10 digits).

4. **Trigger the AI Call:**

Click **Call** next to the candidate.

*(Note: Hunar API has anti-spamguardrails. The call will only go through if it is currently between **08:00 AM and 09:00 PM local time**. Outside of these hours, the call will be queued and scheduled for the next morning).*

5. **Watch the Dashboard:**

Click **View Dashboard**. Wait for the call to finish on your phone. Thanks to your `polling_worker.py` script running in the background, the dashboard will automatically refresh and display the **Recording Link** and **Fit Summary** within seconds!

Troubleshooting

- **"Call already exists and is active"**: You cannot click "Call" twice on the exact same candidate. If you want to test again, create a brand new candidate (e.g., "Test Candidate 2").
- **Dashboard not updating**: Make sure your `python polling_worker.py` script is actively running in a terminal window.
- **Phone not ringing**: Check the time. The Hunar API restricts outgoing calls at night. Try again during daytime business hours!